

The Vault Holds the Liquidity; the Owner Commits the Rules

SoFi: A High-Level Explainer for DeFi Readers

Deterministic Limbo Vaults, singleton exact-output routing, client-computed quorum binding, and non-authoritative storage

Brandon “Cryptskii” Ramsay
Irrefutable Labs Inc.

September 2026

Abstract

SoFi is a market layer built directly on the Deterministic State Machine. A liquidity provider funds a Deterministic Limbo Vault (DLV) by moving value out of its own spendable balance and into encumbered reserve state inside its own DSM state. From that moment the vault is an executable counterparty: a live trader can buy from it while its owner is absent, because the value and the rules under which it may be exchanged were both committed before that trader existed. The owner still owns the reserves, still controls the vault by having written its policies, and is bound by those policies exactly as the trader is.

The central claim is that a decentralised exchange does not need a pool contract, a global transaction order, a mempool, a sequencer, a priority-fee auction, an oracle feed, or a clock. Every trade is verified and re-simulated locally by the trader’s own SDK against a vault parent identified by the hash of its complete state. Exactly one route and one exact output are signed; there is no slippage band. The only shared decision in the entire system is a consume-once binding over one vault parent, computed by the client from an owner-chosen set of five storage members with a fixed threshold of three — and that binding does not move reserves. Reserves move only when the trader’s own ordinary DSM step is accepted and produces verifiable acceptance evidence. Storage nodes persist and serve opaque bytes; they do not price, route, vote, or judge.

This explainer accompanies the SoFi specification [1] and the DSM papers it builds on [2, 3, 4, 5]. It is written for a reader who knows DeFi — automated market makers, order-book and intent-based exchanges, rollup sequencers, oracles, perpetuals — and is organised around that reader’s objections rather than the order in which the mechanism is constructed. Shaded boxes state how the DeFi design solves the corresponding problem and where SoFi differs. Costs and boundaries are stated where they arise.

Contents

I	What Is Claimed, and Who Decides	4
1	The Central Question	4
2	The Entire Idea in One Diagram	5
3	Six Answers Before the Details	6
4	A DLV Is Not a Pool	7

5	Why an Absent LP Can Trade	8
6	The One Shared Decision	9
7	Binding Is Not Realization	9
8	Storage Nodes Are Librarians	10
9	There Is No Mempool to Front-Run	11
II	The Construction	12
10	Vault State and Its Identity	12
11	Owner Authority: Signature or Precommitted Mechanism	13
12	The Trade: Intent, One Route, Exact Output	13
13	The Settlement Bundle	15
14	Client-Driven Quorum Binding	15
15	The Trader Parent Fence	16
16	Trader Acceptance and Realization	17
17	Composition and Its Four Outcomes	17
III	The Owner's Side	18
18	Owner Catch-Up Is Synchronization	18
19	Owner Close	19
20	Multi-Vault Atomicity	20
IV	Markets Without a Clock	20
21	Prices Without an Oracle	20
22	Perpetuals and Liquidation	21
23	Clockless Liveness	21
V	Boundaries, Compromise, and Assessment	22
24	The Online Boundary	22
25	What Goes Wrong, and What It Costs	22
26	What a Compromise Means	23

27 Relation to Everything Else That Works	24
28 What Is Not Introduced, and What Is Not Claimed	24
29 The Complete Causal Architecture	25
30 Final Invariant	26
31 Conclusion	26

Part I

What Is Claimed, and Who Decides

The DeFi reader’s objections, in the order they occur. Everything here is stated normatively in the specification; the mechanism is built in Part II.

1 The Central Question

Every exchange has to answer one question about every trade:

Whose value is moving, and who said it could?

In DeFi the answer has one shape. The value belongs to a contract — a pool, a vault, a margin account — because the liquidity provider deposited it there and took back a claim. The contract “said it could” because the chain placed the trader’s transaction into a global order and the EVM executed the contract’s code at that position. Custody lives in the contract; authority comes from ordering.

SoFi answers differently:

The value belongs to the liquidity provider, and it is still inside the liquidity provider’s own state. The liquidity provider said it could move — in advance, by committing the policy under which the vault executes. No global order is consulted, because the verifier recomputes validity from the vault parent, the route, and the proofs.

The rule everything follows from

$$B_{LP} \longrightarrow R_{DLV}.$$

Funding a vault is a state move, not a promise. Value leaves ordinary spendable balance and enters DLV reserve state in one conservation-preserving transition. There is no second spendable copy on the owner’s side, and there is no deposit receipt that represents the reserves somewhere else. The vault *is* the reserves.

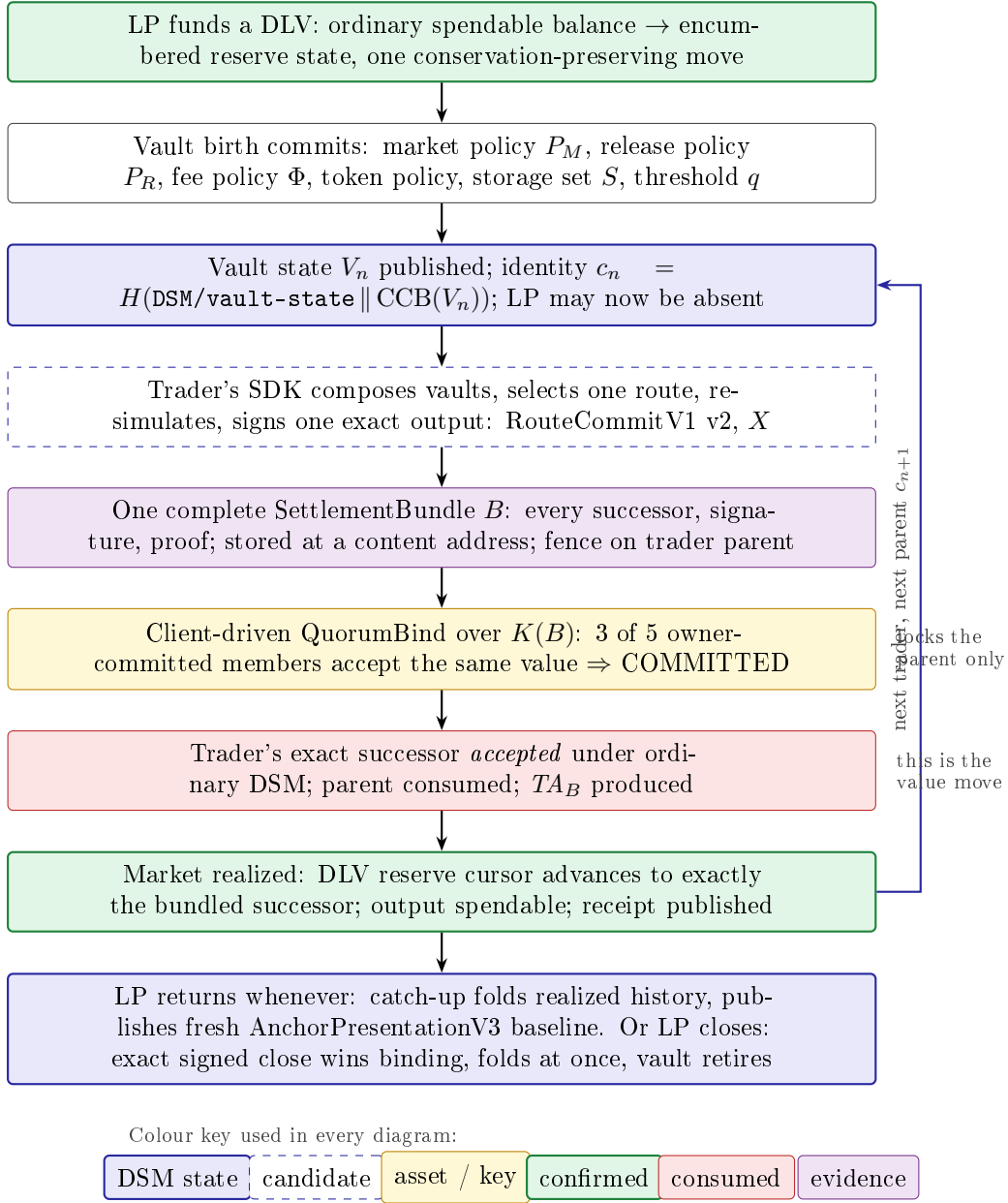
The owner controls the vault but is also bound by it. Ownership does not let the owner reach in and spend the encumbered reserves. A market transition must satisfy the owner-committed market policy, the token policy, the actual remaining reserves, the exact route commitment, and the exact parent identity. Getting the reserves back is itself a governed vault transition,

$$R_{DLV} \longrightarrow B_{LP},$$

under the release policy the owner committed at creation.

Two facts, never conflated *Binding finality* is a historical fact: one candidate was chosen, consume-once, for a vault parent. *Realization* is the reserves actually advancing. In SoFi these are separate, and the second requires the trader’s own bilateral step to have been accepted under ordinary DSM. A trader who wins the binding and walks away has locked a parent. It has not moved a reserve, changed a price, or fabricated an input.

2 The Entire Idea in One Diagram



Three features of that picture are the whole design.

The top band is the owner's only required act. Fund, commit policy, publish. Everything below happens without the owner present, and nothing below can violate what the owner committed above.

The middle band is local. Composition, selection, re-simulation, signing, bundle construction: all of it is the trader's own SDK verifying bytes it fetched. No shared service is consulted for validity.

The one yellow box does one narrow thing. It decides which of possibly several concurrent candidates gets this vault parent. It does not order trades across vaults, does not compute a price, and does not move value. The red box below it is where value moves, and that is an ordinary DSM step in the trader's own chain.

3 Six Answers Before the Details

The question the reader has	The answer, and where it is earned
Where is the liquidity?	Inside the liquidity provider’s own DSM state, encumbered as DLV reserves. Not in a contract, not pooled with strangers’ capital, not represented by an LP token. A large trade can draw from several independent vaults atomically, but each remains its owner’s. (Sections 4, 10)
How does it trade while the LP is away?	The owner committed the market policy, fee policy, size bounds, token constraints, storage set, and threshold when the vault was created. A funded vault <i>is</i> the owner’s authority, embodied as executable state. The trader is online; only the LP is absent. (Sections 5, 11)
What replaces the chain ordering the trades?	Nothing global. Each trader recomputes validity from the vault parent’s complete-state hash c_n , the signed route, and proofs. The only shared decision is a consume-once binding for that one parent, needed because two strangers can target it concurrently. (Sections 6, 14)
Isn’t the “quorum” just a tiny validator set?	No. The five members store opaque bytes and enforce a per-key compare-and-exchange. They do not parse the trade, do not know q , do not talk to each other, and produce no collective decision. The client authenticates responses and counts locally. (Sections 8, 14)
Where is the slippage tolerance?	There is none. The trader signs one route and one exact output. If re-simulation at unlock does not reproduce that output byte-for-byte, the trade rejects. A different route is a fresh quote and a fresh signature, never a fallback hidden in the same commitment. (Section 12)
What about MEV?	There is no mempool, no sequencer, no priority-fee auction, and no ordering surface between disjoint vaults. Price is recomputed from a committed parent, so a stale parent fails before value moves. What an adversary can do is win a binding and walk away, which locks one parent. That is a liveness cost, stated plainly. (Sections 9, 25)

Just enough notation

Vault state V_n , identity c_n .

V_n is the complete vault state at vault-local generation n : owner binding, reserves R_A, R_B , policies P_M, P_R, Φ , encumbrance set E , budget β , local parent commitment h_n , owner authority position, storage set S , threshold q . Its sole identity is $c_n = H(\text{DSM/vault-state} \parallel \text{CCB}(V_n))$, the hash of the complete canonical state.

Route commitment X .

The trader signs one RouteCommitV1 binding one selected route, one parent c_n per hop, one exact output. $X = H(\text{DSM/ext} \parallel RC^*)$ where RC^* is the signature-zeroed commitment bytes.

SettlementBundle B , identifier b .

The complete immutable object the SDK proposes: every DLV successor, the trader successor, every signature, proof, and recovery object. $b = H(\text{DSM/settlement-bundle} \parallel \text{CCB}(B))$.

Resource key k_v .

$k_v = H(\text{DSM/binding-keyset} \parallel c_{n,v})$, one per consumed vault parent; $K(B)$ is the sorted set for the bundle.

Storage set S , threshold q .

Owner-committed at vault birth. The storage profile is $|S| = 5$, $q = 3$.

Trader acceptance TA_B .

The canonical artifact proving the trader's exact successor from the bundled parent was accepted under ordinary DSM, with inclusion evidence under the post-advance root.

Class C, K, N.

Core verifier; SDK that constructs, binds, verifies, and materialises; storage node, non-authoritative.

4 A DLV Is Not a Pool

The word “vault” invites the wrong picture, so it is worth stating what a DLV is not.

It is not a shared pool. Nobody else's capital is in it. It is not contract custody. There is no code address that owns the reserves and no upgrade key that can change how they behave. It is not represented by a claim token. The owner does not hold an LP share redeemable against a pot; the owner holds the vault, and the vault holds the reserves.

What it is: a state object inside the owner's DSM state that holds actual reserve value and acts as a precommitted, executable bilateral endpoint. Funding debits the owner's spendable balance and credits the vault's reserves in one transition:

$$V_n = (g_o, d_o, \text{vault_id}, n, R_A, R_B, P_M, P_R, \Phi, E, \beta, h_n, r_o, S, q).$$

Reserves must not remain simultaneously available as ordinary spendable owner balance. After funding, the reserve value is represented by the DLV and nowhere else.

Aggregation without pooling A single pair conversion may draw from more than one independent vault where the active profile admits it. Each allocation names the vault parent it consumes by c_n :

$$a = (\text{parent_binding}, \Delta_{in}, \Delta_{out}, e, \Phi).$$

The route binds every consumed parent into one SettlementBundle and one binding transaction. It executes fully or not at all. Ten LPs can jointly fill a trade none of them could fill alone, and none of them has handed custody to anything.

DeFi Comparison: the AMM pool

In a constant-product AMM, liquidity providers deposit both assets into one contract and receive LP tokens. The contract owns the reserves. Depth comes from pooling: everyone's capital sits under one address, one invariant, one fee tier. Withdrawing means burning LP tokens against the contract's current balances, at whatever ratio the pool has drifted to.

A DLV is one owner's reserves under one owner's policy. Depth for a large trade comes from a route drawing on several vaults atomically, not from those vaults sharing an address.

Architectural consequence:

No pooled custody, no LP-token layer, no contract that can be upgraded, paused, or drained as a unit. A vault's blast radius is its owner's reserves.

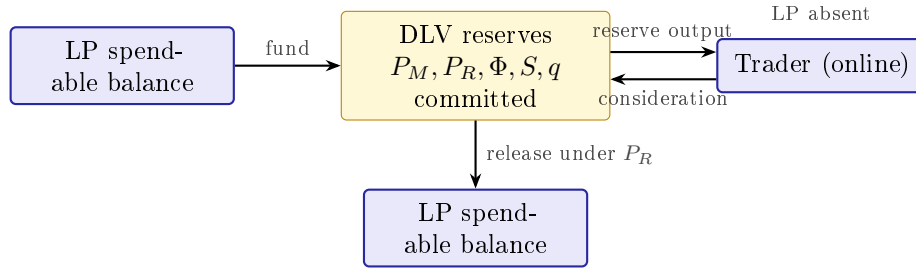
Tradeoff / boundary:

Depth is assembled per route rather than sitting pre-aggregated. Whether a given profile admits multi-vault fanout, and how deep, is a profile decision (Section 20).

5 Why an Absent LP Can Trade

An ordinary DSM relationship has a strict rule about absence. A present party may send its own value toward an absent counterparty, but it cannot make the absent party's value move outward. There is no authority for that, because the absent party has not signed anything about this transition.

A funded DLV is that authority, committed in advance. The value is already inside an executable state object, and the conditions under which it may exchange were fixed at vault birth. When a live trader supplies consideration that satisfies those conditions, the corresponding reserve output moves — not because the owner is consulted, but because the owner already answered.



reserves move only through committed policy, in either direction

The exception is narrow and should be read narrowly. The DLV is an exception *only* to the need for a fresh owner participation step. It is not an exception to parent binding, conservation, the live trader's signatures, Tripwire, token policy, or the vault's own policy. Everything that ordinary DSM checks is still checked; the only thing missing is a fresh signature from someone who already signed the rules.

Key Idea

The owner is bound by the same policy that binds the trader. Ownership of a DLV does not mean the ability to spend its reserves directly. Returning reserves to spendable balance is a governed vault transition under the release policy, consuming the same vault parent that a market trade would consume, and subject to the same contention rules.

DeFi Comparison: passive liquidity

AMMs already let liquidity trade while its owner is away; that is their point. The mechanism is that the owner gave the capital to a contract, and the contract's code is what executes. The owner's absence is possible because the owner no longer holds the asset. SoFi's owner still holds the asset. Absence is possible because the owner precommitted the exchange rule inside their own state, and DSM's verifier enforces precommitted rules without asking the committer again.

Architectural consequence:

Passive market-making with no custody transfer. The reserves never leave the owner's state; what left is the owner's discretion over them, for as long as they remain encumbered.

Tradeoff / boundary:

The owner cannot change the rules of a live vault by a governance vote or an admin call. Changing policy means closing the vault under P_R and opening another.

6 The One Shared Decision

Everything so far is bilateral DSM. Then a public vault creates one problem that bilateral DSM does not have.

A vault is executable by anyone. Two unrelated traders can read the same vault parent c_n and each construct a valid fulfilment before either sees the other. Both candidates are correctly signed, correctly priced, and correctly bound to the same parent. Within one trader's own chain, DSM's Tripwire rejects a second successor to a consumed parent. But these two traders do not share a chain. There is no single history in which their candidates could be linear.

So SoFi adds one scoped mechanism: a consume-once binding over the vault parent's resource keys, decided by a client-driven quorum transaction over the storage set the owner committed. The liquidity provider chooses S and q at vault birth. Reachability at trade time cannot change either. The trader's SDK contacts the members, authenticates distinct responses, verifies the returned canonical bytes, and locally determines whether q qualifying members accepted the same value for the same key set.

The result is a client-assembled binding certificate over one named resource. It is not a replicated transaction log and not a consensus result over the system. It establishes no ordering between unrelated vaults or unrelated trades.

Call it consensus, then ask where the bottleneck is A reader may want to call a three-of-five agreement "consensus." The label is not the point; the bottleneck is. A blockchain's consensus is a throughput ceiling because every transaction in the system passes through one ordering decision. Here there is no such decision. A binding over vault V_1 and a binding over vault V_2 touch different keys, are computed by different traders, and never wait for each other. Ten thousand vaults trading at once are ten thousand independent certificates, each the size of one quorum read, and adding a vault adds a key rather than load on anything shared. What the model achieves is that there is no shared sequencer whose throughput caps the system, and that is the property the label was supposed to be about.

DeFi Comparison: the sequencer

On an L1 the validator set orders everything; on a rollup a sequencer does, usually one operator with a forced-inclusion escape. Every swap on every pool waits for the same ordering decision, and whoever makes it has a reordering surface.

SoFi has no ordering decision across vaults. The only agreement is per vault parent, is computed by the trader who needs it, and settles a single question: which candidate consumes this parent.

Architectural consequence:

Throughput scales with the number of vaults and traders, not with the capacity of one sequencer. There is nothing to decentralise a sequencer *away from*, because there is none.

Tradeoff / boundary:

The binding needs q of the owner's committed members reachable. If the fixed quorum is permanently unavailable, that vault cannot be traded or closed until it is. The threshold cannot be lowered on the fly, by anyone, for any reason.

7 Binding Is Not Realization

Once the client obtains q valid matching acknowledgements, the candidate is *binding-final* for the vault parent: no competitor can consume that parent. This is an objective historical selection fact and it is not undone by anything that happens later.

It does not move the reserves.

For a market bundle, reserves advance only when the initiating trader’s own exact bilateral successor — the one already committed inside B — is accepted under ordinary DSM parent binding, signature, conservation, and Tripwire rules, and produces the trader acceptance artifact TA_B . The verifier obtains TA_B , the settlement receipt evidence, and the inclusion proof. Only then does the DLV reserve cursor realize the bundled successor.

COMMIT $\rightarrow$ parent locked $\rightarrow$ trader accepts own step $\rightarrow TA_B \rightarrow$ reserves advance.

Until TA_B verifies, the SDK must not expose the output as spendable, must not publish a success receipt, must not display the route as successful, and must not fold any market DLV successor from B .

Why the two-phase shape is not a half-completed trade A binding-final market bundle whose trader leg never completes *blocks* the parent. It does not change the advertised reserves, does not create a partially executed exchange, and does not fabricate an input. The old reserves remain the last realized state. The cost of a trader winning and walking away is that one parent is occupied, indefinitely, because the protocol defines no timeout escape — a timeout would be a clock, and no clock enters validity.

Owner close is deliberately different An owner close is one-phase. The exact owner-signed close successor is complete before binding begins, and there is no second sovereign leg waiting to be accepted. If a valid close wins the binding, it folds immediately and retires the vault. Section 19 states why this asymmetry is correct rather than convenient.

Key Idea

Binding finality answers *which* candidate gets the parent. Realization answers *whether* the exchange happened. A market trade needs both; the second is an ordinary DSM step in the trader’s own chain, and the vault only follows it.

DeFi Comparison: atomic execution in a block

An on-chain swap is atomic because the whole thing is one transaction in one block: ordering, execution, and settlement collapse into a single event decided by the chain.

SoFi separates selection from settlement because selection is the only thing that needs to be shared, and settlement is something the trader’s own chain already knows how to do. Nothing is half-done in between: reserves either sit at the old parent or at exactly the bundled successor.

Architectural consequence:

Reserves cannot move on the strength of a storage-layer event. Storage can lock a parent; only an accepted bilateral step can spend one.

Tradeoff / boundary:

A parent can be locked and unrealized. That is the honest price of refusing to let five storage nodes move money, and it is a liveness property, not a safety one.

8 Storage Nodes Are Librarians

The five storage members do three separate jobs, and none of them is judging.

Canonical immutable object store. Exact canonical bytes under a deterministic content address, $\text{addr}(P) = H(\text{DSM/storage-object} \parallel N \parallel H(N \parallel P))$. Re-putting identical bytes is idempotent; different bytes at the same address are refused.

Discovery index. A logical path mapping to one or more content addresses. A “latest” path is a convenience for finding bytes, never an authority about which bytes are current. Validity rules resolve a path, fetch the immutable bytes, and verify them.

Generic binding state. Opaque conditional-binding records,

$$G = (\text{schema}, \text{round}, \text{tx_id}, \text{keyset_digest}, \text{value_digest}, \text{value_addr}, \text{status}),$$

used by client-driven quorum transactions. The node may interpret only the generic fields needed to enforce compare-and-exchange and monotonic rounds. It must not decode the bundle those records point to.

What a member must not do is spelled out as a hard list: compute AMM prices, outputs, fees, or allocations; evaluate intent bounds, owner predicates, or vault economic validity; parse a SettlementBundle to decide storage acceptance; determine q or count other members’ responses; select a route, candidate, or winner; sign anything with financial authority; treat a mutable discovery path as canonical identity; return fabricated bytes under a content address; or claim a complete query while omitting data it holds.

What a member may do is equally narrow: compute storage addresses, enforce immutability, maintain indexes, perform a local compare-and-exchange over opaque keys, keep monotonic round metadata, update a sorted opaque key set atomically inside one local transaction, return binding bytes to recovery, and authenticate its own identity.

DeFi Comparison: validators and data-availability layers

Validators execute and order; DA layers and sequencers publish and order; in every case the infrastructure that holds the bytes also has a say in what they mean. Paying validators is paying for judgement.

SoFi storage members are paid for persistence, bandwidth, availability, indexing, and generic conditional-storage service. They are not paid for ordering, inclusion, computation, or validation, because they do none of it.

Architectural consequence:

Compromising a storage member yields nothing financial: it cannot make an invalid bundle valid, forge a signature, alter a vault invariant, or choose a route. The client detects wrong canonical bytes and treats missing or invalid responses as unavailable.

Tradeoff / boundary:

Availability is a commercial matter. A member may go offline, rate-limit, or drop a subscriber, and that can make a vault’s quorum temporarily unreachable. It cannot change S , q , parent binding, or economic validity.

9 There Is No Mempool to Front-Run

The DeFi reader’s sharpest objection is MEV [7], so it deserves a direct answer rather than a slogan.

A sandwich requires three things: seeing the victim’s intent before it executes, inserting a transaction ahead of it, and having the victim’s execution priced against the state your insertion produced. SoFi removes each.

There is no global pending queue. A trader’s route is constructed locally and its bundle is published to the vault’s storage set at binding time, not broadcast to a network waiting to be ordered. There is no priority-fee market in which to buy the slot ahead of it; storage members define no fee auction and no order between disjoint resource sets. And the price is not “whatever the pool state is when the chain gets to me.” The route is bound to a specific parent c_n , the hash of the complete vault state, and the output is exact. If anyone advances the vault first, the route fails parent binding before value moves; it does not execute at a worse price.

What an adversary *can* do is race for the same parent and win the binding. That gives them the parent, at the committed policy’s price, for their own trade. It gives them no information advantage over the loser, no ability to move the loser’s price, and no ability to extract from the loser’s execution, because the loser’s execution does not happen: it rebuilds against the new parent with a fresh quote.

DeFi Comparison: MEV

The mempool plus priority fees plus a globally ordered execution creates the sandwich: watch, insert, let the victim execute against your state. An industry exists to run this, and the mitigations — private relays, batch auctions, order-flow auctions — accept the ordering surface and try to manage who owns it.

SoFi has no ordering surface to manage. Visibility of a bundle to the storage set grants no priority, and exact-output binding to a named parent means no execution ever happens at a price other than the one the trader signed.

Architectural consequence:

No sandwich, no back-run against your fill, no sequencer that can reorder you.

Tradeoff / boundary:

Same-parent contention is real, and losing it costs a fresh quote. A hostile actor can lock a parent by winning binding and not completing. Both are liveness costs, and neither moves reserves or changes a price.

Part II

The Construction

The objects, identities, and procedures the specification actually defines, in the order they are built.

10 Vault State and Its Identity

The most consequential decision in the design is what a route points at when it names a vault parent. The answer is the hash of the *complete* canonical vault state:

$$c_n = H(\text{DSM/vault-state} \parallel \text{CCB}(V_n)).$$

Every route allocation and every route hop carries this value as **parent_binding**. The vault identifier, the generation, the reserves, the encumbrance set, the policies, the storage set, and the threshold are all fields of V_n , so c_n commits all of them. None of them is an independent parent fact a hop could state separately and get wrong.

The local parent chain is

$$h_0 = H(\text{DSM/vault-state-parent/genesis/v2} \parallel \text{vault_id}), \quad h_n = c_{n-1} \ (n > 0),$$

so each vault state names its predecessor by that predecessor’s full identity, exactly as ordinary DSM state is hash-adjacent to its parent.

Why the complete state, and not a projection A parent identity built from selected fields of V_n — a generation, a reserves digest, an encumbrance commitment carried alongside — is a second object that can disagree with the state it stands for, and every such object is a place where a route and a vault can diverge. Hashing the complete canonical state leaves nothing to

carry separately and nothing to reconcile. The owner’s authenticated baseline, `AnchorPresentationV3`, signs only $H(\text{DSM/vault-state-anchor/v3} \parallel 0x00 \parallel c_n)$; any convenience metadata sent alongside is rederived from V_n by the consumer, who refuses on disagreement.

Stale construction If a realized successor advances a vault from c_n to c_{n+1} , any route still bound to c_n fails parent binding before value moves. A stale route is *unavailable*, not a trade that completed and then failed. Since the route commitment is a singleton, staleness does not fall through to another route; it requires a fresh quote, fresh composition, fresh `RouteCommitV1`, fresh signature, and fresh X .

Key Idea

Stale-state rejection and same-parent contention are different mechanisms. A stale route fails because its hop’s `parent_binding` is not the current c_n ; that is a local check. Two live constructors over the same current c_n are resolved by quorum binding; that is the shared check. Conflating them is the most common misreading of the design.

11 Owner Authority: Signature or Precommitted Mechanism

A successor V_{n+1} carries owner authority in exactly one of two ways. Either the witness contains the owner’s signature over $\text{CCB}(V_{n+1})$, or the witness contains an owner-signed fulfilment mechanism committed *before* V_{n+1} existed, plus proof that V_{n+1} satisfies every bound of that mechanism.

The market path uses the second form. The owner-committed market fulfilment mechanism is

$$M = H(\text{DSM/fulfillment} \parallel c_0 \parallel \text{CCB}(B_M)),$$

where c_0 already commits the vault id, fee policy, storage set, threshold, owner authority position, and birth state, and B_M carries only the additional market-exercise bounds that do not already live in V_0 , such as the per-transition size ceiling and the authorised encumbrance purposes.

The close path uses the first form. The live implementation reconstructs the close operation from the composed frontier and verifies the owner’s SPHINCS+ signature over it. The bundle’s “close shape” is a discriminator that tells the verifier which authority form to check; it is not itself authorization.

Public data grants no reserve control. Holding the vault advertisement, every state, every receipt, and every storage replica gives a party the ability to *construct* a candidate against the vault. It does not give them the ability to make that candidate satisfy P_M with an input they did not supply, or to satisfy P_R without the owner’s signature.

12 The Trade: Intent, One Route, Exact Output

A trade begins as a route request committing the input token, output token, input amount, maximum hop count, and a nonce. The SDK then does the work that a DEX router and a chain would do between them, entirely locally:

1. query discovery indexes for candidate vault addresses; fetch canonical bytes by content address;
2. verify each candidate; compose each surviving candidate through its successor-kind gates (Section 17); exclude any whose current composed state cannot be established;
3. build the liquidity graph from established candidates only, and select one best route under deterministic rules;
4. stamp each hop with its composed parent c_n ;

5. construct one unsigned RouteCommitV1, sign it, and derive the external commitment $X = H(\text{DSM}/\text{ext} \parallel RC^*)$.

Singleton, exact, no slippage band RouteCommitV1 binds exactly one route and one exact expected output. The committed route set is $R = \{\text{selected_route}\}$. It carries no fallback routes, no floor on final output, and no minimum output. If unlock-time re-simulation does not reproduce the exact expected output, the trade rejects. There is no runtime interval within which a worse fill is accepted.

Two conforming SDKs given the same vault states produce byte-identical routes. That is a consequence of canonical commit bytes (fixed-width big-endian integers, length-prefixed strings, sorted sets, explicit absence markers, no floating point) and checked integer arithmetic throughout.

Pricing For the constant-product exact-input family, with input a against reserves x, y and fee f in basis points, the output is one fused expression with one floor division:

$$\text{out} = \left\lfloor \frac{y \cdot a \cdot (D - f)}{x \cdot D + a \cdot (D - f)} \right\rfloor, \quad D = 10,000.$$

The full input a is credited to reserves; the fee remains in reserves as liquidity-provider yield. Conservation is checked per token across one atomic execution:

$$\sum_i R_{i,t}^{(n+1)} + \sum \text{out}_t = \sum_i R_{i,t}^{(n)} + \sum \text{in}_t - \text{fee}_t.$$

Encumbrance A claim against a vault parent is

$$e_j = H(\text{DSM}/\text{enc-claim} \parallel p \parallel \text{claim_seq} \parallel \text{amount} \parallel \text{token} \parallel \text{purpose}), \quad p = c_n.$$

The encumbrance set E is a field of V_n , so c_n commits it. A claim is consumed at most once, and for every token $\sum_{e \in E_t} \text{amount}(e) \leq R_t$. Priority among internally valid claims is a policy question inside one transition; it is never the quorum winner rule.

DeFi Comparison: routers and slippage

A DEX aggregator finds a route, quotes an output, and submits a transaction with a minimum-output guard. Between quote and execution the chain may reorder; the guard exists because the price you get is whatever the pool holds when your transaction lands. Fallback paths and slippage tolerance are how the router survives the ordering it does not control.

SoFi's router is SDK-resident, deterministic, and unprivileged. It binds to a named parent and signs an exact output. Nothing between signing and execution can change the price, because the route is only valid against the parent it named.

Architectural consequence:

Quoted output equals executed output, or the trade does not execute. An external router can suggest a suboptimal route but cannot bypass intent bounds, re-simulation, parent binding, quorum binding, or conservation.

Tradeoff / boundary:

Any change in the vault state between quote and binding is a failed route, not a slightly worse fill. Active vaults under contention will cost traders more fresh quotes. The specification chooses rejection over tolerance on purpose.

13 The Settlement Bundle

Before any shared step begins, the SDK constructs one complete immutable SettlementBundle:

$$B = (\text{version}, \text{storage_set_id}, q, I, X, \text{selected_route}, \text{trader_parent}, \text{trader_successor}, \\ \{T_v\}_{v \in \text{route}}, \{P_v\}_{v \in \text{route}}, \text{signatures}, \text{recovery_material}),$$

with identifier $b = H(\text{DSM/settlement-bundle} \parallel \text{CCB}(B))$. For each consumed vault v , the parent side of its transition T_v is the exact c_n of the consumed state; vault id, generation, state commitment, and reserves digest are read from the authenticated V_n that c_n identifies.

Complete means complete. B contains every DLV successor, the trader successor, every signature, witness, proof, and recovery object needed to verify the selected route and to recover the binding decision without any fresh private material from the constructor. If the constructing device dies after publishing B , a recovery driver can finish the binding transaction from B alone.

B is stored at $\text{addr}(B) = H(\text{DSM/storage-object} \parallel \text{namespace} \parallel b)$, and the SDK verifies the content address before binding. The bundle consumes the sorted resource key set $K(B) = \{k_v : v \in B\}$, one key per vault parent.

14 Client-Driven Quorum Binding

QuorumBind is the SDK procedure that drives a crash-recoverable transaction over exactly the S , q , and $K(B)$ that B commits. It is client-driven in the literal sense: the nodes do not call one another and do not determine quorum.

Each member exposes a generic **CompareExchangeMany**: given a strictly sorted opaque key set, an expected digest of the prior record set, and replacement record bytes, either all named local keys change to the replacement or none do. The operation is atomic *within one member*. It knows nothing of q , contacts no peer, and does not decide whether the caller committed a trade.

The SDK counts only distinct authenticated members of the owner-committed S . A response from outside S , a duplicate identity, or bytes that fail verification is not countable. Nobody — not the SDK, not a node, not a helper that “derives” a threshold — may lower q because a member is slow, offline, or quota-limited.

The storage profile $|S| = 5$, $q = 3$. Any two quorums intersect in at least $3 + 3 - 5 = 1$ member, which is the minimal intersection needed for uniqueness under durable member records. It is the strict majority of five and carries no durability margin beyond that one-member intersection; the number was chosen for uniqueness, not for Byzantine tolerance, and the specification says so.

Outcomes

COMMITTED(B). At least q distinct members accepted the same safe transaction value for the complete key set. B is binding-final for those parents. For markets, this does not advance reserves.

ABORTED(B). Recovery safely decided this proposal will never become binding-final.

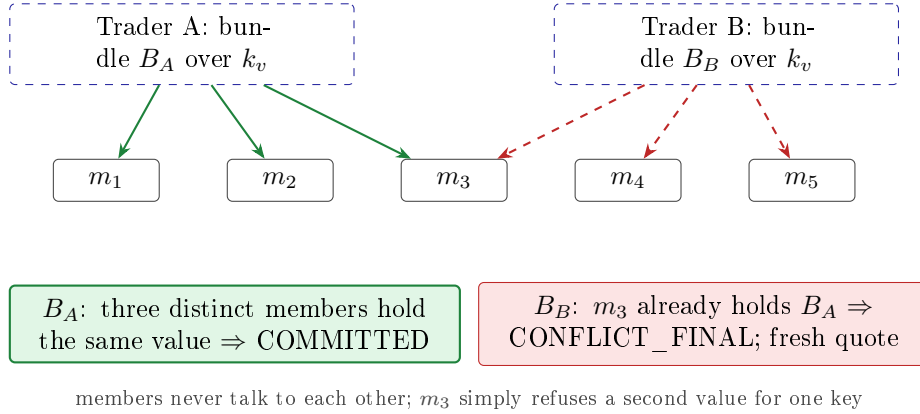
CONFLICT_FINAL(B'). Recovery established that an overlapping key already belongs to a different binding-final transaction.

INVALID. Canonical or storage-domain checks failed before a value could be chosen.

Two further states are protocol-visible and are *not* ABORT: RECOVERING (the transaction is being driven to a terminal outcome) and INDETERMINATE (a mutating step was issued and the outcome was lost). An indeterminate transaction is durable client state. It is never silently converted to ABORT by a restart, a timeout, a UI dismissal, or a route search. Pending or

accepted generic records may remain on members after a constructor crash; that persistence is what makes recovery possible and it is not finality.

The same-parent safety statement Assume $q > n/2$; each member’s compare-exchange and multi-key update are atomic and durable; recovery obeys the safe-value rule; and counted member identities are distinct and authenticated. Then two different bundles with overlapping $K(B)$ cannot both be chosen binding-final for the same vault resource by conforming quorum transactions. If two distinct binding-final bundles are ever observed for one parent, storage safety has failed; SoFi permits no tie-break and quarantines the lineage for operator recovery.



DeFi Comparison: what “finality” means

Chain finality is a property of a global history: after enough blocks or enough signatures, this block’s order will not change, and every transaction in it inherits that.

SoFi binding finality is a property of one key set: this candidate, and no other, consumes these vault parents. It says nothing about any other vault, any other trade, or any global sequence, and it does not say the trade happened.

Architectural consequence:

Finality is scoped and cheap: one quorum read per trade, computed by the party that needs it.

Tradeoff / boundary:

There is no global view to consult. A third party learns what happened to a vault by walking its composed state from an authenticated baseline (Section 17), not by reading a ledger.

15 The Trader Parent Fence

Before the first *mutating* binding operation for a market bundle, the SDK durably records a local fence:

$$F_B = (\text{trader_chain_id}, \text{trader_parent_state_commitment}, b, \text{tx_id}).$$

If the fence cannot be persisted, QuorumBind must not begin.

The fence exists because the trader’s side is ordinary DSM, and ordinary DSM would otherwise let the trader do what ordinary DSM allows: advance from that parent with some other successor. That would be safe for the trader’s own chain (Tripwire would accept the first, reject the second) but catastrophic for the vault, which may have already been bound to a bundle naming a trader successor that now can never be accepted. So while the transaction is RECOVERING or INDETERMINATE, no different successor may be created, accepted, or used as a later parent from the fenced trader parent. If the bundle commits, the fence permits only

the exact trader successor already inside B , and releases only when that successor is accepted through ordinary bilateral advancement. A fresh intent or nonce does not bypass it.

Key Idea

An unresolved market settlement fences the initiating trader's parent, not the whole trader. The trader's other relationships are unaffected. What is frozen is exactly the parent that a possibly-binding bundle has already claimed.

16 Trader Acceptance and Realization

Market realization requires two things: binding finality for B , and a valid trader acceptance artifact TA_B . At minimum TA_B binds:

1. the ordinary DSM accepted-successor commitment C_T^+ ;
2. the ordinary successor-state authentication σ_T^+ over C_T^+ ;
3. the exact route commitment X and bundle identifier b ;
4. a deterministic settlement/acceptance leaf under the post-advance root R_T^+ committed inside C_T^+ ;
5. an inclusion proof under that authenticated root.

A conforming verifier parses and verifies C_T^+ under ordinary DSM rules, checks that it commits the exact bundled trader parent and successor, verifies σ_T^+ directly over C_T^+ , extracts R_T^+ , verifies the inclusion proof, and checks that the leaf binds exactly $(\text{trader_parent}, \text{trader_successor}, b, X)$.

TA_B is not a storage statement and not a second trader authorization. It packages the ordinary acceptance the trader's chain already performed, in a form a stranger can check. Because B is bound before TA_B exists, the trader publishes a non-authoritative locator under `sofi/trader-acceptance/base32(b)`; the verifier follows it to content-addressed bytes and verifies them against the exact bundle, trusting the path for nothing.

Receipts A successful receipt is $\text{SofiReceipt} = (b, X, a_B, \{(\text{successor_hash}_v, \text{witness_hash}_v)\})$ with $\rho_B = H(\text{DSM/sofi-receipt/v1} \parallel \text{CCB}(\text{SofiReceipt}))$, published together with $\text{CCB}(B)$ and $\text{CCB}(TA_B)$. The receipt is evidence and an index. It creates no binding finality, no acceptance, no realization, and no reserve-provenance authority. A third-party verifier follows it to B and TA_B and checks every fact independently. Certification creates the obligation to publish; publication failure does not undo realization.

17 Composition and Its Four Outcomes

Nobody hands a trader the “current” vault state. The trader establishes it by *composing*: starting from the latest authenticated owner baseline (an `AnchorPresentationV3` over some c_k) and folding every verified successor that names the current composed parent, in order, until nothing more verifies.

The gate for folding a successor depends on its kind. A market successor folds only when its complete bundle is binding-final *and* the exact initiating trader successor has verified TA_B . An owner release/close successor folds when its exact candidate is binding-final for the current parent and verifies under release policy, token policy, conservation, and the owner's concrete signature.

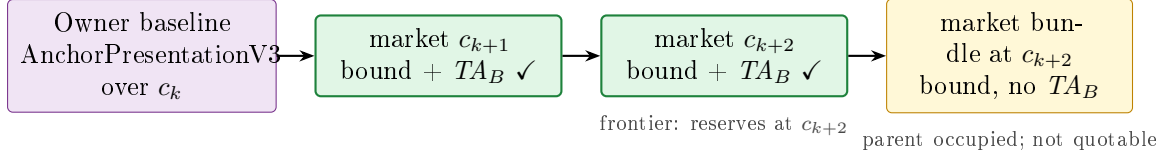
The walk keeps four outcomes distinct, and the distinction is the whole reason a locked-but-unrealized parent is safe:

Free. The quorum explicitly reports no binding at the parent key. This parent is the frontier unless the trader's own local unresolved fence overlays it.

Bound and realized. A binding-final bundle exists and its completion evidence verifies. Fold and continue.

Bound and unrealized. A binding-final market bundle exists but TA_B , receipt, or proof material is absent. The old reserves remain the last realized state and the parent is occupied. Stop here; do not quote against it.

Unavailable or invalid. Evidence is missing, contradictory, or malformed. Fail closed. This vault does not enter the liquidity graph.



Select, verify, build, bind — the full order For one selected route the SDK: composes every named vault; rejects the route if any hop's parent binding is stale; verifies proofs, signatures, encumbrances, owner authority, and predicate inputs; re-simulates every allocation and leg; enforces exact output and fees; constructs every exact signed successor and recovery proof; constructs one complete B ; stores it and confirms the content address; persists the fence; invokes one QuorumBind over $K(B)$; on COMMITTED completes the trader successor through ordinary DSM, verifies TA_B , and only then marks the market realized; on ABORTED or CONFLICT_FINAL folds no successor and requires a fresh quote; on RECOVERING or INDETERMINATE keeps the trader parent fenced and drives the transaction to a terminal outcome.

Part III

The Owner's Side

What the liquidity provider sees when it comes back, how it gets its reserves out, and why a close behaves differently from a trade.

18 Owner Catch-Up Is Synchronization

While the owner is absent, its *local* relationship state against the vault lags the vault's realized market history. Trades happened; the owner's device does not yet have them folded. When the owner returns to an active vault, it applies certified market successors in causal order.

Each apply is synchronization of already-realized history. It is not a second authorization, not a veto, not an ordering step, and not a second value move. It gates nothing: no trader settlement, no market realization, no composition by anyone else, no future admission, no QuorumBind, no fence release, no close finality. The trades were real before the owner looked.

After catch-up the owner's authenticated state publishes a fresh AnchorPresentationV3 over exactly the certified frontier V_k . That baseline shortens everyone's future composition walk. It does not move trade value again.

The cost that is not magically bounded While a vault stays active, composition work between authenticated owner baselines grows with realized market history. An owner who never returns leaves every trader walking a longer chain from an old anchor. Catch-up collapses that history into a new baseline. The specification states this as a real cost rather than hiding it.

DeFi Comparison: LP accounting

A pool contract updates its reserves in every swap, in place, on chain. The LP's share is a token whose redemption value drifts with the pool. The LP does not “catch up” because the LP never held the state; the contract did.

A SoFi owner holds the state and catches up to what its own vault did under its own committed policy. The mechanism is the same bilateral synchronization DSM uses for any relationship where one party was away.

Architectural consequence:

Fee yield accrues inside the vault's own reserves, under the owner's own authority, with no claim token to price or redeem.

Tradeoff / boundary:

An absent owner imposes composition cost on traders. Returning and publishing a fresh anchor is how an owner keeps its vault cheap to quote.

19 Owner Close

Owner close consumes the same vault parent resource that a market trade would consume, and uses the same client-driven quorum-binding machinery. Two consequences follow, and they run in opposite directions.

Close does not jump the queue If a market candidate is already indeterminate or binding-final for the current parent, owner close is not an escape hatch. The SDK must recover that prior market outcome and must not bypass it. A trader who won binding and has not yet produced TA_B blocks the owner from closing, for as long as that remains unresolved. Beta defines no timeout for it.

But a winning close folds immediately If the owner's valid close candidate reaches binding finality first, the result differs from a market win. The release policy P_R is owner-local: its truth is decidable from the authenticated current parent, the exact owner-signed close successor, the token policy, and canonical bytes that all exist before binding begins. There is no second sovereign leg waiting to be accepted, because the owner's signature over the exact successor *is* the completion witness. Binding finality therefore realizes the close at once. Both reserve leaves drain to zero in one conservation-preserving owner/DLV update, and the vault retires. A closed vault cannot be quoted, funded, traded, or closed again, and it needs no separate catch-up baseline.

Key Idea

The asymmetry is principled. A market bundle has two sovereign legs — the vault's and the trader's — and the second must be accepted before the first realizes. A close has one sovereign leg, already signed. Waiting for a second acceptance that cannot exist would be a rule with nothing to wait for.

DeFi Comparison: removing liquidity

Burning LP tokens is a contract call; it executes when the chain includes it, against whatever the pool's reserves are at that position, and it competes with swaps in the same ordering.

A SoFi close competes only with candidates on its own vault parent, and the owner gets back exactly the reserves at that parent, because nobody else can advance the parent once the close is binding-final.

Architectural consequence:

No impermanent-loss surprise between deciding to withdraw and the withdrawal landing: the close names the exact parent it drains.

Tradeoff / boundary:

A close can be blocked by an unresolved market binding on the current parent, indefinitely. The specification chooses that over any timeout that would let a clock decide whether a trader’s claim on the parent still counts.

20 Multi-Vault Atomicity

Where a profile admits a route consuming multiple vault parents, the route is one Settlement-Bundle and one quorum-binding transaction over the complete key set $K(B)$. Intermediate generic records on fewer than q members do not materialise value. No verifier may accept a subset of hops as successful execution of the original route.

A profile states which route shapes it admits; a verifier certifies only the shapes it can independently verify, and the route-wide binding model holds for every admitted shape.

Part IV

Markets Without a Clock

Reference prices without an oracle, perpetuals without a funding clock, and liveness without a timeout.

21 Prices Without an Oracle

Every verified execution produces a trade digest committing the pair, executed amounts, the parent identity c_n of every participating vault, fees, and X :

$$d = H(\text{DSM/digest} \parallel \text{CCB}(\text{TradeDigest})).$$

A digest must not contain a global height, a predecessor digest, a shared sequence position, or a “latest” claim. The collection of digests for a pair is a *set*. Nobody orders it, because nothing about a digest is a position.

A reference window is a finite canonical set of admissible digests,

$$W = H(\text{DSM/ref-window} \parallel \text{pair_id} \parallel \text{Canon}(\{d_i\})).$$

For an odd admissible member count $w \geq m$, $\Pi(W)$ is the median of the members’ prices. If the window is undersized or any member fails the required filter, $\Pi(W)$ is undefined and any predicate that consumes it fails closed.

A bilateral transition that consumes a reference commits W and $\Pi(W)$ in its successor; the counterparty independently verifies the named window and signs or refuses. A unilateral branch commits its admissibility filter rule before execution — counterparty distinctness, size floor, reserve-ratio bounds, positive-fee reality, capital and depth constraints — and the advancing party supplies the window and proofs while the Core recomputes the rule.

DeFi Comparison: oracles

On-chain protocols that need a price outside their own pool import it: a push oracle network signs a feed, or a TWAP is read from a pool over a block window. In both cases the price is an authority the protocol trusts, and both depend on chain time.

SoFi’s reference price is a median over a set of executed, verified trade digests that the consuming party selects and the counterparty checks. There is no feed to trust and no clock window; an undersized or contaminated window is simply undefined.

Architectural consequence:

No oracle operator, no oracle-manipulation surface via a single pool’s spot, and no dependence on block timestamps.

Tradeoff / boundary:

A reference is only as good as the admissibility filters committed for it, and a thin market may not produce a definable window at all. Failing closed is the designed behaviour, not a bug.

22 Perpetuals and Liquidation

Perpetual instruments are DLV predicate families on the same settlement machinery; they introduce no protocol clock or epoch. Funding may be expressed as a deterministic function of co-signed changes in admissible trade evidence rather than elapsed time. Let $D(W)$ be the digest-member set of a window; the activity delta is

$$\delta_W = |D(W_{n+1}) \setminus D(W_n)|,$$

with a per-transition clamp preventing unbounded work or one-shot accumulation.

Liquidation is a bounded fulfilment branch the position holder committed at open. A liquidator supplies the required reference evidence and proves the branch predicate; the holder need not be online. Liquidation of any market reserve leg still obeys parent-state, encumbrance, conservation, and quorum-binding rules.

DeFi Comparison: perp DEXes

Perpetual venues pay funding on a schedule (every hour, every eight hours) and liquidate through keepers who watch an oracle and race to be first. Both mechanisms are clocks: one for accrual, one for who gets the liquidation.

SoFi denominates funding in trade activity and precommits the liquidation branch at open. The liquidator’s job is to present evidence that a predicate the holder already signed now holds.

Architectural consequence:

No funding timer to game, no keeper race, no liquidation ordered by whoever saw the oracle tick first.

Tradeoff / boundary:

Activity-denominated funding behaves differently from time-denominated funding in quiet markets, and the position holder must accept a liquidation predicate at open that no one can later soften.

23 Clockless Liveness

No protocol object, predicate, commitment, settlement rule, or validity decision in SoFi may depend on wall-clock time, elapsed duration, a timestamp, a global height, or a shared sequence across independent parties. Where a branch needs timeout-like behaviour, it uses a state-local iteration budget, $\beta_{n+1} = \beta_n - 1$ only on accepted transitions in the relevant local chain. Failed verification, node retries, unavailable routes, and failed binding attempts do not consume budget.

Operational retry timers may schedule work. Elapsed time never changes COMMIT, ABORT, conflict, realization, or fence release.

Safety is independent of time entirely. Liveness is conditional on eventual communication with the fixed quorum and on recovery proposers going quiet. Under unbounded asynchronous delay, or permanent loss of the fixed quorum, the protocol may fail to terminate; overlapping key-set transactions can also fail to terminate while independent recovery drivers keep superseding one another's rounds. This is the FLP boundary [8], and SoFi sits on the safety side of it deliberately: backoff may reduce livelock but never enters signed or hashed state and never decides a winner.

Part V

Boundaries, Compromise, and Assessment

Where SoFi stops, what happens when things break, how it compares to what exists, and what is not claimed.

24 The Online Boundary

A SoFi market trade is an online transaction. The liquidity provider may be absent; that is not the same as the transaction being offline. New market acquisition, recovery of unresolved quorum transactions, and independent establishment of binding evidence may require network access to the committed storage set.

DSM's offline and bearer paths remain governed by their own rules. SoFi defines no portable offline market-settlement witness, and a product must not advertise offline SoFi market settlement. An offline bearer allocation does not become SoFi-tradable liquidity by virtue of what asset it carries; the dBTC explainer [5] states the same boundary from the asset's side.

25 What Goes Wrong, and What It Costs

The specification names its failures; a reader should be able to name them too.

Event	Outcome
Vault advances before binding	Route fails parent binding. Unavailable, not partially executed. Fresh quote.
Two traders, same parent	At most one COMMITTED. The other sees CONFLICT_FINAL and re-quotes against the new parent.
Constructor crashes before any mutating step	Only immutable proposal bytes exist. Nothing is bound. Trader parent was not yet fenced, or the fence is simply released on recovery finding no binding.
Constructor crashes after a mutating step	Recoverable generic records remain on members. Transaction is RECOVERING or INDETERMINATE; trader parent stays fenced until a terminal outcome is established.
Crash after the value was chosen	Binding-final selection exists even though the constructor never saw success. Market still needs TA_B ; close follows its one-phase rule.
Binding-final, trader never completes	Parent blocked. Reserves unchanged, no output spendable, owner close blocked. No timeout escape.
Fixed quorum permanently unreachable	Vault cannot be traded or closed until reachable. Threshold cannot be lowered.
Overlapping recovery drivers livelock	May fail to terminate; backoff is operational only. No winner is decided by time.
Two binding-final bundles for one parent	Storage safety has failed. No tie-break. Lineage quarantined for operator recovery.
Storage member returns wrong bytes	Detected by content-address verification; treated as unavailable. No financial effect.

The protocol-visible failure taxonomy is fixed:

- NO_ADMISSIBLE_ROUTE
- HOP_UNAVAILABLE
- SETTLEMENT_CONFLICT
- SETTLEMENT_RECOVERING
- SETTLEMENT_INDETERMINATE
- SETTLEMENT_UNRESOLVED
- DLV_BINDING_EVIDENCE_UNAVAILABLE
- ENCUMBRANCE_CONFLICT
- PREDICATE_REJECTED
- STALE_STATE
- BUDGET_EXHAUSTED
- PROOF_INVALID
- REFERENCE_UNDEFINED
- WINDOW_REJECTED
- SIZE_BOUND_EXCEEDED
- STORAGE_SAFETY_VIOLATION
- UNSUPPORTED_ROUTE_COMMIT_VERSION
- PARENT_BINDING_MISMATCH

A gate failure is diagnostic: the implementation fixes the cause rather than bypassing the gate.

26 What a Compromise Means

Storage member A member has no financial authority. It cannot make an invalid bundle valid, alter a vault invariant, forge an owner or trader signature, choose a route, or calculate economic finality. The safety model is crash/asynchrony safety under conforming SDKs and authenticated distinct member identities; a malicious member may lie, selectively fail, or return wrong bytes, and each of those is detected or treated as unavailability.

Router An external router may suggest a suboptimal route. It cannot bypass intent bounds, route membership, deterministic re-simulation, parent binding, quorum binding, trader acceptance, or conservation. Default routing is SDK-resident.

Vault owner Compromise of an owner’s signing authority is outside SoFi’s cryptographic protection. An attacker with the owner’s key can close the vault under P_R . Recovery can limit future damage; it cannot rewrite binding-final or realized history. The security of a vault is the security of its owner’s DSM custody, no better.

27 Relation to Everything Else That Works

Constant-product AMMs [6] Pooled custody in a contract, execution at whatever position the chain assigns, slippage tolerance as the survival mechanism, MEV as the tax. SoFi keeps the invariant and the fee-in-reserves yield model, and removes the pool, the ordering, the tolerance, and the surface.

Order-book DEXes A book needs a matcher, and the matcher is either an off-chain operator or a chain’s validator set running the book. Either way, someone sequences. SoFi has no book; liquidity is precommitted policy, and the only contention is per vault parent.

Intent and solver systems Batch auctions and filler networks remove the user from the mempool by handing the intent to solvers who compete to fill it. This is an improvement in who owns the ordering surface. SoFi’s “intent” is a route request the trader’s own SDK resolves deterministically; there is no solver to trust and no auction to win.

Rollup DEXes Cheaper, faster, and still sequenced by one operator per rollup with an escape hatch. SoFi is not a rollup: there is no L1 to settle to, no state root to post, and no sequencer to decentralise.

Perpetual venues Off-chain books or pooled-counterparty vaults, timed funding, keeper liquidations, oracle dependence. SoFi’s perps are predicate families over the same vault machinery, with activity-denominated funding and precommitted liquidation.

	Custody	Who orders trades	Slippage band	Price authority	Clock in validity
AMM pool	contract	validators / sequencer	yes	pool spot	yes
Order-book DEX	contract or venue	operator or validator set	limit price	book	yes
Intent / solver	contract, solver escrow	solvers, then chain	yes	solver quote	yes
Rollup DEX	contract on L2	sequencer	yes	pool spot	yes
Perp venue	pooled or venue	operator / validators	yes	oracle	yes
SoFi	owner’s own DSM state	nobody; per-parent consume-once binding	none; exact output	median over verified digests	none

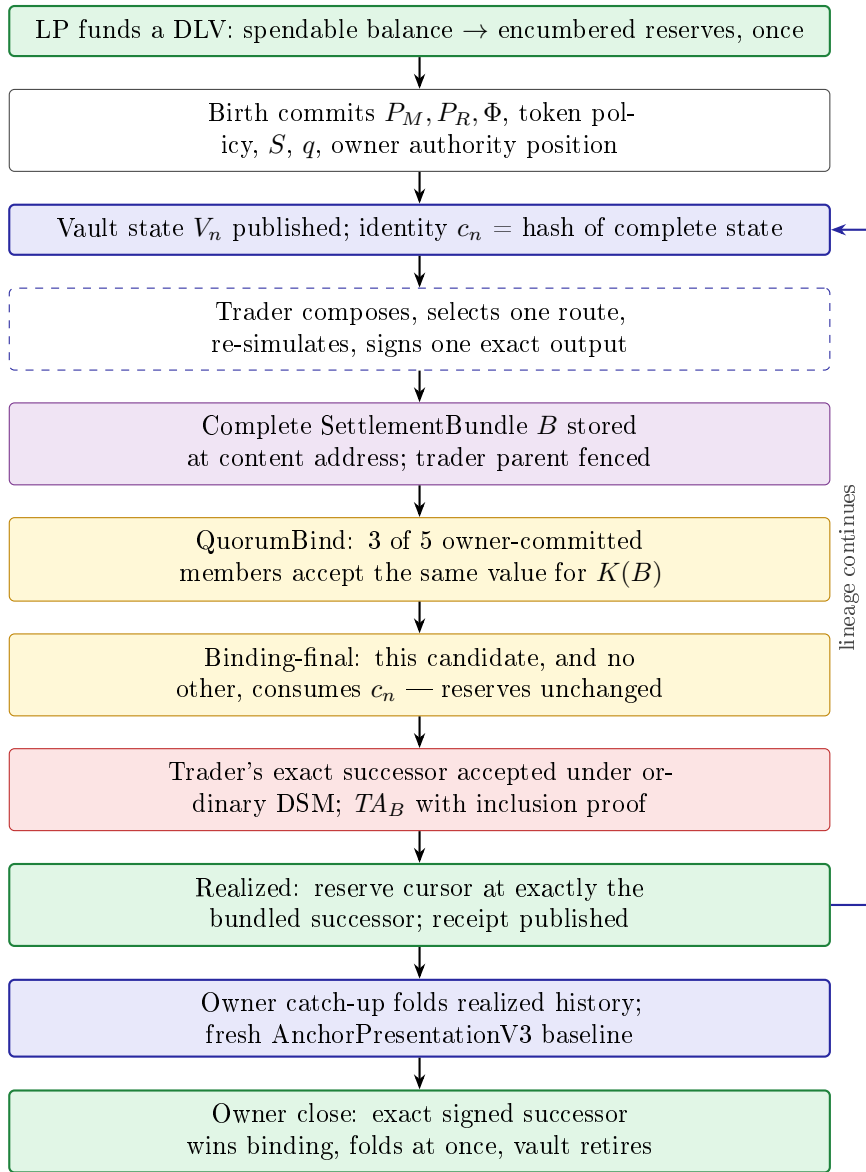
28 What Is Not Introduced, and What Is Not Claimed

Deliberately not introduced No pool contract. No LP token. No validator set. No sequencer. No mempool. No priority-fee or gas auction. No global trade order. No oracle feed as authority. No wall-clock or block-height in any validity rule. No timeout that releases a bound

parent. No storage-node vote on economics. No second trader-side state machine. No pre-signed fallback route inside a route commitment. No slippage interval. No parent identity other than the hash of the complete vault state.

Deliberately not claimed That a market trade is offline. That a bound parent will always eventually realize. That liveness survives permanent loss of the fixed quorum. That overlapping recovery cannot livelock. That three-of-five tolerates Byzantine members beyond crash and asynchrony under authenticated identities. That an owner-offline composition walk is bounded without the owner returning. That a compromised owner key cannot close the vault. That a receipt is authority. Or that conformance follows merely because the primitives exist in a repository.

29 The Complete Causal Architecture



30 Final Invariant

The complete market realization relation, compactly:

$$\begin{aligned} \text{Realized}(B) \implies & \text{BindingFinal}(B, K(B)) \wedge \text{ParentBound}(B, \{c_{n,v}\}) \\ & \wedge \text{RouteCommitV1}(RC) \wedge X = H(\text{DSM}/\text{ext} \parallel RC^*) \\ & \wedge \text{ExactOutput}(B) \wedge \text{Conserved}(B) \wedge \text{PolicySatisfied}(B, P_M, \Phi) \\ & \wedge \text{Accepted}_{\text{DSM}}(\text{trader_successor}) \wedge \text{Verify}(TA_B, b, X). \end{aligned}$$

And for close:

$$\text{Closed}(V_n) \implies \text{BindingFinal}(B_{\text{close}}, k_v) \wedge \text{Sig}_{\text{owner}}(\text{CloseSuccessor}) \wedge P_R \wedge \text{Conserved}.$$

For any parent c_n : at most one binding-final bundle; reserves at c_n or at exactly the bundled successor, never between.

The vault holds the liquidity.

The owner commits the rules and is bound by them.

Binding chooses; only an accepted step moves value.

state $\rightarrow$ policy $\rightarrow$ route $\rightarrow$ bundle $\rightarrow$ binding $\rightarrow$ acceptance $\rightarrow$ realization

31 Conclusion

SoFi is bilateral finance with executable encumbered reserve state. The liquidity provider funds a vault by moving value into reserves it still owns, and the vault becomes an executable endpoint under policies the owner committed before any trader arrived. The trader can buy from that state while the owner is absent because both the value and the authority to exchange it are already inside the vault.

The route model is exact and singleton. One signed RouteCommitV1, one external commitment X , one parent identity c_n per hop derived from the complete canonical state. Any different route is a fresh quote and a fresh signature.

The one shared decision is scoped to one vault parent and computed by the party who needs it, from five owner-committed storage members and a threshold of three. Storage persists and serves; the SDK counts and verifies; the Core evaluates economics.

Binding finality, trader realization, owner catch-up, and owner close are kept separate on purpose. A market bundle that wins binding without verified trader acceptance blocks the parent without moving reserves. A bundle with both realizes exactly the successor it committed. Owner catch-up records what already happened. A valid close that wins binding folds immediately and retires the vault.

If two binding-final bundles are ever established for one parent, storage safety has failed, and SoFi does not retroactively pick a winner. The construction therefore preserves what DSM already guarantees: state is authority; validity is independently verified; consumed state does not remain spendable; storage is non-authoritative; and value moves only after the deterministic predicate holds.

References

- [1] B. Ramsay. *SoFi: Sovereign Deterministic Finance — A Normative Specification for Deterministic Limbo Vaults, Encumbrance Accounting, Singleton Exact-Output Routing, Quorum Binding, and Non-Authoritative Storage Infrastructure*. Irrefutable Labs Inc., September 2026.

- [2] B. Ramsay. *Deterministic State Machine: A Concise, Post-Quantum Specification*. Irrefutable Labs Inc.
- [3] B. Ramsay. *Deterministic State Machines as Guarded Linear Constraint Systems*. Irrefutable Labs Inc., 2026.
- [4] B. Ramsay. *Deterministic State Machines Explained Mathematically*. Irrefutable Labs Inc., 2026.
- [5] B. Ramsay. *The dBTC Is the Asset; the Bitcoin Key Is Machinery — dBTC: A High-Level Explainer for Bitcoin Readers (DSM-NATIVE dBTC V1)*. Irrefutable Labs Inc., September 2026.
- [6] H. Adams, N. Zinsmeister, D. Robinson. *Uniswap v2 Core*. 2020.
- [7] P. Daian et al. *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges*. 2019.
- [8] M. Fischer, N. Lynch, M. Paterson. *Impossibility of Distributed Consensus with One Faulty Process*. JACM, 1985.